NFL Picks Pool — Implementation Plan

NFL Picks Pool — Full Automation Plan

Context

Ryan currently runs a 40–50 player NFL picks pool (`$50` buy-in, 25k starting points, max 3 ATS bets/week, 18-week regular season + playoffs) manually:

- **Wednesday**: emails the week's slate + spreads
- **Saturday**: manually runs `saturday.R` to pull Google Form responses, generate picks PNGs, save updated `pick_log.csv`
- **Tuesday**: manually runs `tuesday.R` to grab scores via `nflreadr`, settle bets, write `week_log.csv`
- **Wednesday morning**: runs `wednesday.R` to publish standings + new spreads

This requires Ryan's hands every week and outputs static PNG images shared via email. The goal is to **fully automate the pool** with a free-to-host web app so players submit picks online and view live standings without logging in, while Ryan's only ongoing work is recruiting friends and confirming Venmo payments.

Target launch: 2026 NFL season kickoff (Sept 2026, ~4 months away). Ship the full feature set for week 1.

Architecture (decided with Ryan)

Layer	Choice	Why
Frontend + API	FastAPI + Jinja2 + htmx + Tailwind, deployed to Vercel (Python runtime)	Python end-to-end, server-rendered (mobile-friendly), htmx polling for live leaderboard
Database	Supabase Postgres (free 500MB)	Built-in row-level security, easy cron via Edge Functions, real-time channels available later
Scheduled jobs	GitHub Actions (free 2,000 min/mo) running Python scripts	Avoids Vercel's 10-second function timeout for heavier settlement jobs; talks directly to Supabase
Email	Resend free tier (3,000/mo)	Well under our 50 players × 3 weekly emails (~600/mo)
Spread data	The Odds API free tier (500 req/mo)	Cron pulls Wed morning; spreads frozen at publish
Live scores	ESPN public scoreboard API (free, unofficial)	60-sec polling during game windows
Final scores	`nfl_data_py` (Python port of nflreadr)	Authoritative weekly settlement Tue morning
Auth	Custom passwordless magic link table	No Supabase Auth — viewers never log in; players use bookmarked personal link
Raspberry Pi	Not used	Free serverless schedulers cover everything; removes single-point-of-failure

Product Decisions (from clarifying Q&A)

- **Viewing**: 100% public, no login required, any link visitor sees live standings + picks + spreads
- **Picks submission**: Personal persistent magic link per player (e.g., `pickspool.com/p/abc123def`), bookmarkable for the whole season, optionally one-click delivered in the Wed email
- **Player onboarding**: Manual via admin (Ryan adds name + email → system generates link → emailed)
- **Buy-in tracking**: Manual ledger; Ryan toggles "paid" in admin when Venmo arrives
- **Pick validation**: Real-time — form blocks submissions that exceed balance, aren't in 500 increments, or are below 500; shows live running total + remaining points
- **Pick edits**: Allowed until **per-game kickoff** (each game locks independently)
- **TNF rule change** ⚠️ : Current rulebook says picking TNF locks ALL your picks at TNF kickoff. New behavior: only the TNF pick locks; others stay editable until their own kickoffs. Update 2026 rulebook to reflect this.
- **No-bet penalty**: Auto-applies escalating -5k × (consecutive misses + 1) at Sat noon, with an admin "waive" button for emergencies
- **Cancellations**: System auto-detects POST/CANC games from ESPN status, sends Ryan a confirmation prompt, voids on his approval (refunds all bets per rule 3a/3c)
- **Eliminated players** (0 points): Stay in standings, sorted last, marked ELIMINATED — preserves social ribbing
- **Multi-season history**: Out of scope for v1 (current season only). Historical data in `Historical_Results/` stays archived.
- **Spread movement after Wed lock**: Ignored — frozen at publish for fairness
- **UI**: Mobile-first

Database Schema (Supabase Postgres)

```
players (
  id uuid pk,
  name text,
  email text unique,
  magic_token text unique,        -- the /p/<token> in their URL
  paid_buyin boolean default false,
```

```
  starting_points int default 25000,
  created_at timestamptz,
  is_active boolean default true
)

games (
  id uuid pk,
  season int,                    -- 2026
  week int,                      -- 1..22 (incl. playoffs)
  espn_event_id text,            -- for live score lookups
  home_team text,
  away_team text,
  favorite_team text,
  spread numeric,                -- positive value, FAVORITE is favored by
        this many
  kickoff_at timestamptz,
  home_score int null,
  away_score int null,
  status text,                   -- scheduled | in_progress | final | voided
  voided_reason text null
)

picks (
  id uuid pk,
  player_id uuid fk,
  game_id uuid fk,
  pick_team text,                -- 'FAVORITE' or 'UNDERDOG'
  pick_amount int,               -- multiples of 500
  submitted_at timestamptz,
  locked_at timestamptz null,    -- snapped to game kickoff_at when game
        locks
  unique(player_id, game_id)
)

settlements (
  id uuid pk,
  pick_id uuid fk unique,
  result text,                   -- 'win' | 'loss' | 'push' | 'voided'
  net_profit int,                -- + or - amount, 0 for push/voided
```

```
  settled_at timestamptz
)

penalties (
  id uuid pk,
  player_id uuid fk,
  week int,
  amount int,                    -- negative; -5000, -10000, etc.
  waived boolean default false,
  reason text,
  applied_at timestamptz
)

admin_audit_log (
  id uuid pk,
  action text,                   -- 'edit_pick' | 'void_game' |
        'adjust_balance' | etc.
  payload jsonb,
  performed_at timestamptz
)
```

Derived view `standings_v`: aggregates starting_points + sum(settlements.net_profit) + sum(penalties not waived) per player, grouped by week, for fast leaderboard reads.

File / Module Layout

```
/api                           # FastAPI app (Vercel Python serverless)
  main.py                      # FastAPI entry, route mounting
  routes/
    public.py                  # GET / (leaderboard), /week/<n>, /game/<id>,
/player/<token-or-id>
    picks.py                   # GET/POST /p/<token>  -- magic-link picks form
    admin.py                   # GET/POST /admin/*  -- HTTP-basic gated
  templates/                   # Jinja2 + htmx fragments
    base.html, leaderboard.html, picks_form.html, admin/*.html
  static/                      # tailwind output css, logos cache
```

```
  lib/
    db.py                     # Supabase client wrapper
    settlement.py             # Port of tuesday.R logic (pandas)
    spreads.py                # Port of wednesday.R schedule/lines logic
    auth.py                   # Magic token validation, admin basic auth
    email_send.py             # Resend client + template renderers

/jobs                         # GitHub Actions Python scripts
  pull_spreads.py             # Wed 08:00 ET — fetch Odds API, write games
rows, send Wed email
  send_reminders.py           # Fri 20:00 ET — email players who haven't
submitted
  lock_and_reveal.py          # Sat 11:59 ET — lock picks, send "picks live"
email
  poll_live_scores.py         # Every 60s during game windows — update games
table
  settle_week.py              # Tue 09:00 ET — final scores from nfl-data-py,
write settlements + penalties
  detect_cancellations.py     # Hourly Wed–Tue — flag POST/CANC games, alert
Ryan

/.github/workflows/
  cron-*.yml                  # one file per scheduled job above

/migrations/
  001_init.sql                # schema above
  002_seed_2026_players.sql   # populated from admin or one-time CSV import

/vercel.json                  # Python runtime config + Vercel Cron entries
for short jobs
```

Key Behaviors

Magic-link picks flow

1. Ryan adds a player in admin → server generates 32-char token, inserts row, calls Resend with welcome email containing `https://pickspool.com/p/<token>`
2. Player visits link any time → sees current week's games, current balance, prior picks (if any)
3. Form uses htmx to revalidate balance on every change; submit button greys out for invalid states
4. POST writes/updates `picks` rows; each pick's `locked_at` is set when `now() >= games.kickoff_at`
5. Form remains editable until each game's individual kickoff

Live leaderboard

- Server-rendered HTML at `/` includes htmx `hx-trigger="every 60s"` polling on the leaderboard `<table>` fragment
- Fragment endpoint reads from `standings_v` view
- During game windows the `poll_live_scores.py` cron updates `games.home_score/away_score`, which feeds an in-flight "implied result" calc shown alongside final-settled standings

Settlement (Tue morning)

- `settle_week.py` pulls final scores from `nfl-data-py`, computes ATS winner per game, writes `settlements` rows, applies penalties for missed bets, advances `players.starting_points` carryover for next week
- Idempotent: re-running is safe, will only write missing settlements

Admin overrides (all selected)

- Edit picks/bets retroactively (writes audit log entry)
- Adjust player point balances (writes audit log entry)
- Void/restore games + correct game results
- Resend magic link
- Mass-message (in-app banner + email broadcast)

Automated emails (Resend)

- **Wed AM**: "Week N spreads live + last week's results" — to all players
- **Fri 8pm ET**: "You haven't picked yet" — only to players with no `picks` rows for current week
- **Sat noon ET**: "Week N picks are live" — to all players, links to public picks page
- (Tue results email is intentionally skipped — that content is folded into the Wed email)

Critical files (existing) to reference

- `Historical_Code/wednesday.R:24–83` — schedule + spread/line formatting logic to port to `pull_spreads.py`
- `Historical_Code/saturday.R:130–167` — picks aggregation (team_sum, picks_by_team) to port to public display + reveal job
- `Historical_Code/tuesday.R:35–110` — ATS winner calc, push handling, the join logic between spreads/scores — directly portable to `settlement.py`
- `Historical_Code/tuesday.R:115–160` — pool settlement + net_profit + carryover to next week → core of `settle_week.py`
- `Historical_Code/saturday.R:285–318` — no-bet penalty escalation logic → core of `lock_and_reveal.py` and `settle_week.py`
- `Rules/2025 NFL PICKS POOL RULES.pdf` — canonical rules; needs minor update for the new per-game-lock TNF behavior

---

Build Phases (4 months → Sept 2026 kickoff)

Month 1 (May–early June 2026): Foundation - Supabase project + schema migrations - FastAPI skeleton on Vercel with public landing page - Magic-token auth + picks form (no validation yet) - Admin: add player, edit balance, audit log

Month 2 (June 2026): Data pipeline - Port `wednesday.R` spread logic to `pull_spreads.py`; wire The Odds API - Port `tuesday.R` settlement to `settle_week.py`; wire nfl-data-py - GitHub Actions cron wired up + tested against prior-season data (replay weeks from `Historical_Results/Archive_2025`) - Real-time pick validation in form

Month 3 (July 2026): Live + polish - ESPN live score poller + htmx leaderboard refresh - Picks-by-team heatmap, weekly biggest winner/loser callouts - Per-game lock enforcement - Cancellation detection + admin confirm flow - Penalty auto-apply + admin waive - All 3

automated emails via Resend with HTML templates

Month 4 (August 2026): Hardening & launch - Backfill admin: import 2026 player roster - End-to-end dry run with the August preseason schedule - Mobile UX pass + accessibility - Update 2026 rulebook PDF for new per-game-lock TNF behavior - Send player onboarding emails ~2 weeks before week 1

Verification

Local dev - `uvicorn api.main:app --reload` for FastAPI + htmx hot-reload - Local Supabase via `supabase start` (Docker) for isolated dev DB - Each cron job runnable as `python jobs/<script>.py --week N --season 2026 --dry-run`

Replay testing (best signal before launch) Use historical data from `Historical_Results/Archive_2025/` to replay an entire prior season through the pipeline: 1. Seed `players` from the 2025 roster 2. For each week 1–22: run `pull_spreads.py` with mocked Odds API returning the actual published lines (from `line_log.csv`), inject the actual picks (from `pick_log.csv`), run `settle_week.py`, and compare final `standings_v` against the actual `week_log.csv`. They should match exactly.

End-to-end manual checks - Generate a magic link → visit in incognito → submit picks → confirm DB rows and re-edit until lock - Trigger each GitHub Action manually via `workflow_dispatch` and verify email delivery + DB writes - Admin: void a fake game, confirm settlements refund correctly, audit log captures it

Launch readiness checklist - Vercel + Supabase + Resend + Odds API + GH Actions all on free tiers with usage monitoring - A working backup script that dumps Supabase to a private GitHub repo nightly - Documented runbook for: “spread didn’t import,” “ESPN API returned bad score,” “magic link broken for player X”